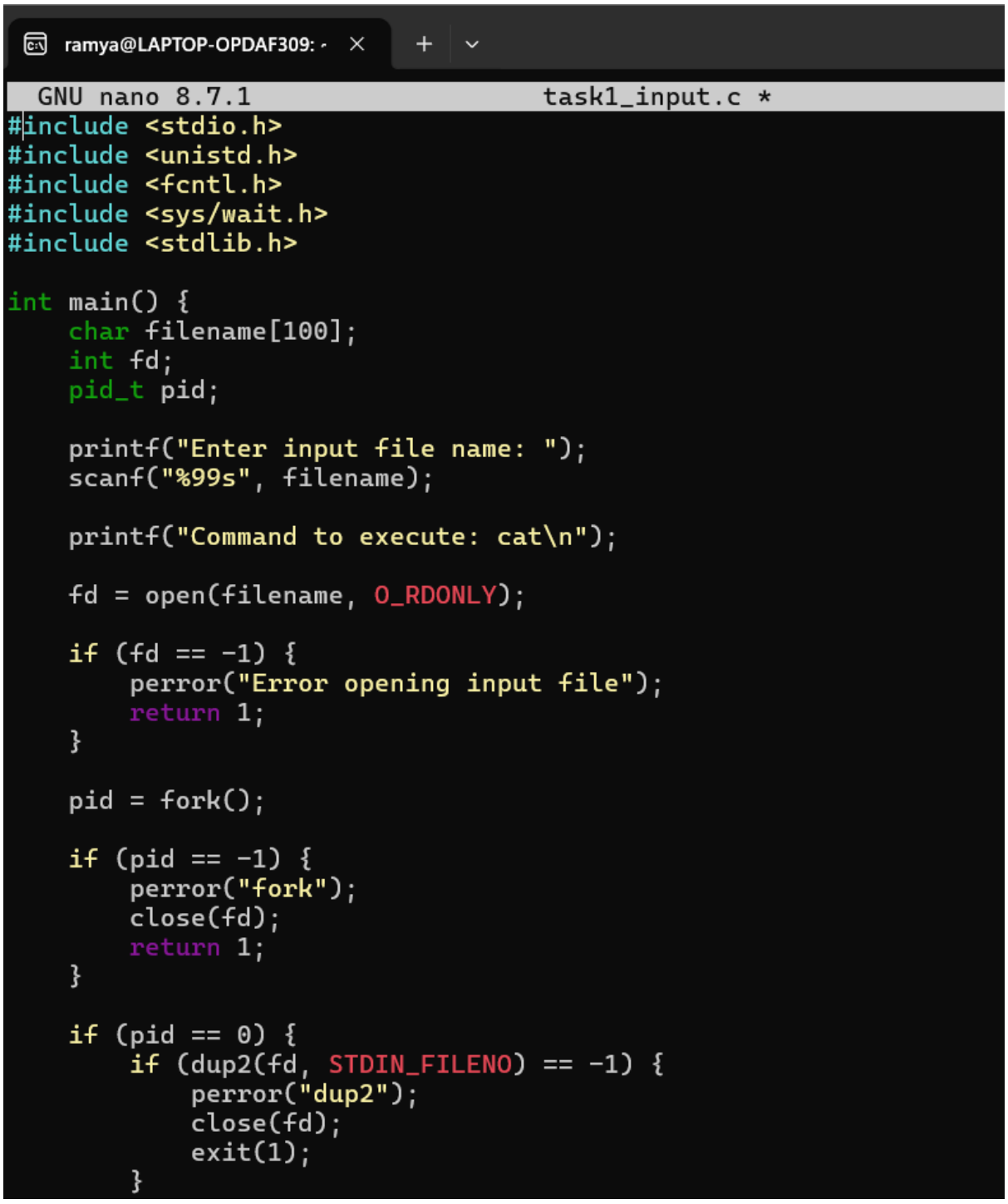
WEEK – 7

TASK 1 : To parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs



```
    close(fd);

    waitpid(pid, NULL, 0);

    printf("Input redirection completed.\n");

    return 0;
}
```

```
ramya@LAPTOP-OPDAF309:~/skill7$ nano task1_input.c
ramya@LAPTOP-OPDAF309:~/skill7$ gcc task1_input.c -o task1_input
ramya@LAPTOP-OPDAF309:~/skill7$ ./task1_input
Enter input file name: input.txt
Command to execute: cat
Hello Linux
Input redirection test
Input redirection completed.
ramya@LAPTOP-OPDAF309:~/skill7$ ./task1_input
Enter input file name: missing.txt
Command to execute: cat
Error opening input file: No such file or directory
```

TASK 2 : To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection

ramya@LAPTOP-OPDAF309:

GNU nano 8.7.1 task2_output.c *

```c
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    char filename[100];
    int fd;
    pid_t pid;

    printf("Enter output file name: ");
    scanf("%99s", filename);

    fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd == -1) {
        perror("Error opening output file");
        return 1;
    }

    pid = fork();

    if (pid == -1) {
        perror("fork");
        close(fd);
        return 1;
    }

    if (pid == 0) {
        if (dup2(fd, STDOUT_FILENO) == -1) {
            perror("dup2");
            close(fd);
            exit(1);
        }
        close(fd);

        execlp("ls", "ls", NULL);

        perror("Command execution failed");
        exit(1);
    }

    close(fd);
    waitpid(pid, NULL, 0);

    printf("Output redirection completed.\n");

    return 0;
}
```

```
ramya@LAPTOP-OPDAF309:~/skill7$ nano task2_output.c
ramya@LAPTOP-OPDAF309:~/skill7$ gcc task2_output.c -o task2_output
ramya@LAPTOP-OPDAF309:~/skill7$ ./task2_output
Enter output file name: output.txt
Output redirection completed.
ramya@LAPTOP-OPDAF309:~/skill7$ cat output.txt
input.txt
output.txt
task1_input
task1_input.c
task2_output
task2_output.c
ramya@LAPTOP-OPDAF309:~/skill7$ ls -l output.txt
-rw-r--r-- 1 ramya ramya 75 Sep 28 03:34 output.txt
ramya@LAPTOP-OPDAF309:~/skill7$ echo "OLD DATA" > output.txt
ramya@LAPTOP-OPDAF309:~/skill7$ cat output.txt
OLD DATA
ramya@LAPTOP-OPDAF309:~/skill7$ ./task2_output
Enter output file name: output.txt
Output redirection completed.
ramya@LAPTOP-OPDAF309:~/skill7$
```

TASK 3 : To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios

ramya@LAPTOP-OPDAF309:

GNU nano 8.7.1 task3_append.c *

```c
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    char filename[100];
    int fd;
    pid_t pid;

    printf("Enter file name: ");
    scanf("%99s", filename);

    fd = open(filename, O_WRONLY | O_CREAT | O_APPEND, 0644);

    if (fd == -1) {
        perror("Error opening file");
        return 1;
    }

    pid = fork();

    if (pid == -1) {
        perror("fork");
        close(fd);
        return 1;
    }

    if (pid == 0) {
        if (dup2(fd, STDOUT_FILENO) == -1) {
            perror("dup2");
            close(fd);
            exit(1);
        }
```

```
        close(fd);

        execlp("ls", "ls", NULL);

        perror("Command execution failed");
        exit(1);
    }

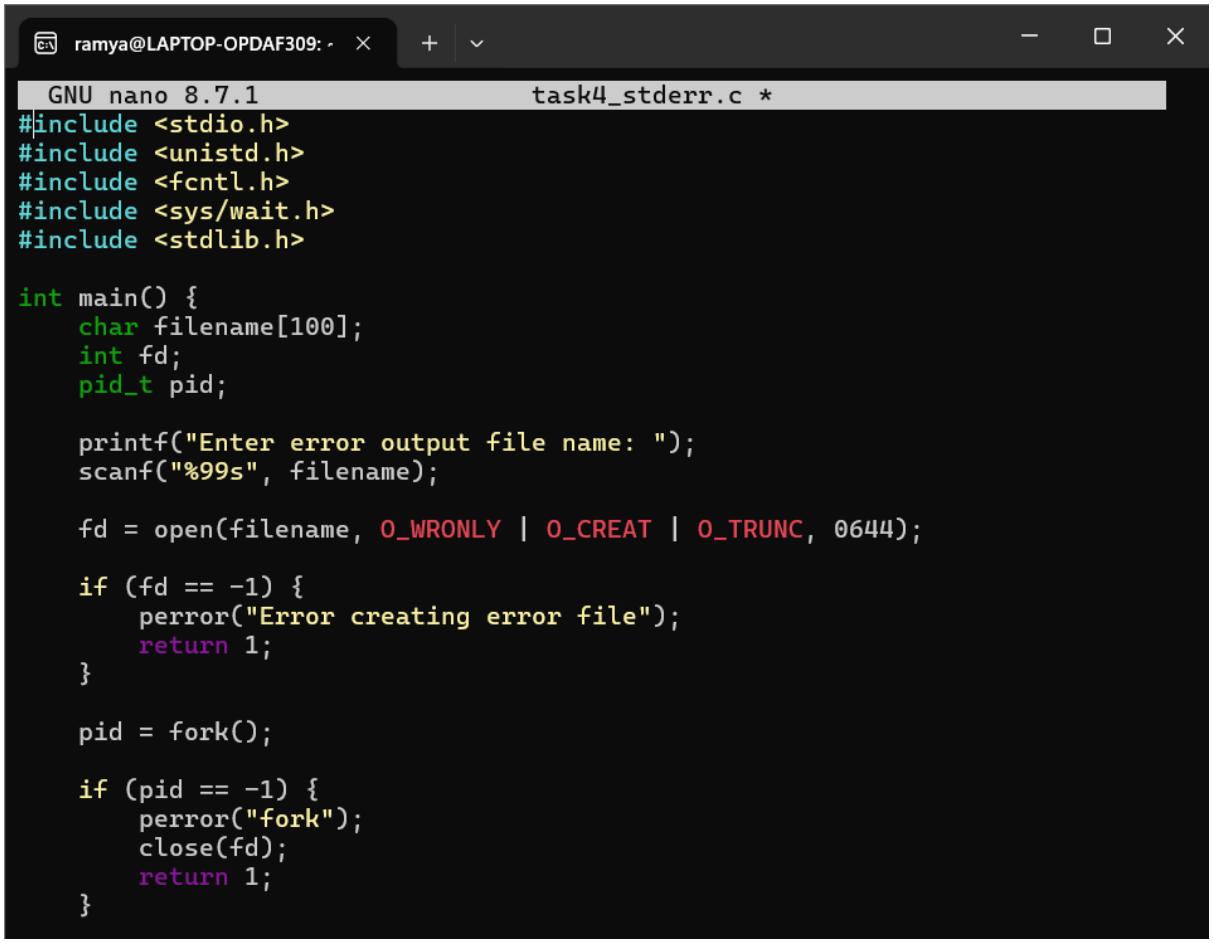
    close(fd);
    waitpid(pid, NULL, 0);

    printf("Append redirection completed.\n");

    return 0;
}
```

```
ramya@LAPTOP-OPDAF309:~/skill7$ nano task3_append.c
ramya@LAPTOP-OPDAF309:~/skill7$ gcc task3_append.c -o task3_append
ramya@LAPTOP-OPDAF309:~/skill7$ echo "Existing data" > append.txt
ramya@LAPTOP-OPDAF309:~/skill7$ cat append.txt
Existing data
ramya@LAPTOP-OPDAF309:~/skill7$ ./task3_append
Enter file name: append.txt
Append redirection completed.
ramya@LAPTOP-OPDAF309:~/skill7$
```

TASK 4 : TO parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Case




```
    if (pid == 0) {

        if (dup2(fd, STDERR_FILENO) == -1) {
            perror("dup2");
            close(fd);
            exit(1);
        }

        close(fd);

        execlp("ls", "ls", "file_does_not_exist.txt", NULL);

        perror("Command execution failed");
        exit(1);
    }

    close(fd);

    waitpid(pid, NULL, 0);

    printf("stderr redirection completed.\n");

    return 0;
}
```

```
ramya@LAPTOP-OPDAF309:~/skill7$ nano task4_stderr.c
ramya@LAPTOP-OPDAF309:~/skill7$ gcc task4_stderr.c -o task4_stderr
ramya@LAPTOP-OPDAF309:~/skill7$ ./task4_stderr
Enter error output file name: errors.txt
stderr redirection completed.
ramya@LAPTOP-OPDAF309:~/skill7$ cat errors.txt
ls: cannot access 'file_does_not_exist.txt': No such file or directory
```